

Lec 61 Generative Pre-trained Transformer(GPT) Decoder-only Large Language Model

Lec 61 Generative Pre-trained Transformer(GPT) Decoder-only Large Language Model

Lecture	61 / 63 + intro
Duration	40:53
Captions	English (auto-generated)
Words	6,032
Video	_zH-VPjiHfM
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome back to the lecture series on fundamentals of generative AI and large language models. In my previous session, we have discussed about BERT. In today's session, we will be speaking about generative pre-trained transformers which we call as the GPT. It's a decoder only large language model. Okay. So the topic of discussion today is GPT and we will see how the contents of today's session goes like. So the contents are evolution of GPT, need for GPT1, why GPT is decoder only architecture, auto regressive text generation by GPT, GPT learning framework, GPT u generative pre-training unsupervised fine-tuning, GPT architecture, the decoded only transformer, large language model and limitations of GPT. Okay. So

[00:57] these are the contents for today's discussion. So we will start with the evolution of GPT how over a period of time the GPTs they evolved. So if we see in 2018 GPT1 was first you know introduced and in the same year BERT was also introduced. So we will look into how this BERT and GPT these are two different you know language models how they are you know different what exactly what is the work that is

performed by bird and what is the work that is performed by GPT we will have a look and then 2019 GPT2 came into picture 2020 GPT3 then 3.5 in 2022 GPT4 came into picture in 2023 then GPT4 OMI that 2024 it came then GPT5 in 2025 and now GPT 5.6 six. So you know it's going on now. So over a period of time

[01:59] even though it is introduced in 2018 the first GPT architecture over a period of time they have improved their person you know performance the scale at which they can you know process um text and uh the multimodal um performances. So all these things over a period of time till today it is you know going uh you know advancing the technology is advancing. Now we will have a look what is the difference. Bird was also introduced which is also a language model. GPT1 also was introduced in 2018 which is also a you know decoder only language model. So what is the difference between them? So BERT basically it is primarily designed for language understanding.

[02:45] This we have seen in our previous lecture also. There was no text generation that was happening from birth whereas only the understanding part of the uh you know sentence was more important there. Whereas in GPT1 it was focused on language generation. So sentence generation paragraph generation like that. Bird is encoder only transformer and this is decoder only transformer. So we will see how these two architectures how they are different and how they enable. We have already seen how birth is enabled with the decoder only encoder only architecture for text understanding for language understanding. In this session we will see how GPT only with the with the decoder only transformer architecture

[03:32] how it is you know um generating the sentences. Okay. Then Bert predicted masked tokens. So in Bert we have seen there is a masked word which was predicted by Bert during the pre-training process like the boy is playing football. So it will take the context from both the sides. So the boy is so that means it is it is something in the present tense and then there is a masked word in between and football is there at the end. So football is related to something in the sports category related to something playing play and this is term that the boy is you know it is it will tell that it is uh playing is in the um present tense. So the boy is playing football. So like this in the B model we have seen how masked word

[04:24] prediction happens or masked to token predictions happen. Whereas GPT predicts the next token because it is generating some sentence. GPT which is based on decoder architecture. It predicts the next token in the sentence or next word in the sentence. Okay. Now why was GPT1 developed? What is the reason for that one? Okay. Same thing there are some uh limitations in traditional you know natural language processing uh you know language models that we are uh using before B and GPT came into picture. It required labelled data sets for any NLP task you need to train a model it required label data sets and existence of labeled data set was limited building separate models for every NLP task was expensive. So these prompted you

know

[05:21] the uh development of GPT and it came into picture in the uh year 2018 by OpenAI a group of scientists from OpenAI a Radford and his team and the name of the paper is improving language understanding by generative pre-training. So this is the paper and with this one there is this groundbreaking work of GPT that started and it is continuing till today. Okay. So what is the key idea of GPT1 which made it so different from the other language models? So pre-train one language model on large amount of unlabeled text using next word prediction. So the key takeaway from this point is you take you know a model pre-train it and how do you pre pre-train it? It is with unlabeled text. So that is where

[06:21] you are actually coming out of this requirement of label data set. So this is self-supervised. So you don't require label text. And how do you train the GPT? It is trained using next word prediction. Okay. So you give a sequence of word tell GPT to predict the next word. You have the target word and you know you see the error and you know uh keep training it improving the performance with the reduction of the in the error. Okay. Now once you have pre-trained your model [clears throat] using unlabelled text for next word predictions using unlabelled text uh you know and the process that by which you train it is um next word prediction then you fine-tune the same model for different NLP task instead of building a separate one. So

[07:15] that is similar to bird. So you have you first pre-train it and how do you pre-train it? you have unlabeled data set and you use next word prediction process for pre-training and then use that one for downstream tasks. Downstream tasks when I tell it can be sentiment classification, it can be um uh natural language inferences, it can be question answering anything. Okay. So you have a model pre-trained which is having all the language understanding and it knows the sequence in which you know words should come. it has already learned that one and then you fine-tune it for downstream tasks. Okay. Now why is GPT a decoder only architecture? Now when we while we were studying uh transformers we have already discussed

[08:03] that encoder is m majorly for understanding the language and decoder is mainly used when you need to generate the language. Okay. So GPT uses decoder only transformer because text generation requires predicting next word. If you need to generate text like you know uh you know paragraphs generation and other things. So you require predicting the next word using the word that already been seen by the model. So that is where your decoder comes into picture. If you need to predict a next uh uh a text, you need to predict the next word using the words that already have been seen by the model.

[08:48] Okay. So that is what is done in decoder. We have seen it and that generates the text one token at a time using auto regressive next token prediction. So this is a new term auto regressive next token prediction. What it is we will see in my

next slide. So it generates the decoder generates one token at a time. This is very important. And this prediction or generation depends on whatever tokens it has already seen. Based on that one only the next token will be generated. Okay. And the process by which this generation happens is is a auto regressive process which we will see in my next slide. And it uses casual masked self attention to hide the future words. This is very important. we use casual masked uh self

[09:37] attention or we call it as masked self attention in the decoder. Why this masked self attention is required? Because I don't want my model to look into the future words that are coming. Okay. So that is how the next word prediction happens. So the model will see only the cat sat on the mat. If the sentence is the cat sat on the mat on the dash, I want my model have already seen you know cat sat on the now based on this whatever it has seen before it will predict this word. There will be masking of this word. So there it cannot see the words that are coming later also. Not only this one if it is like the cat is jumping on the jumping above the wall.

[10:34] Okay. So if that is another sentence that is there coming after this prediction point my model will not be able to see any of these things. So these are all masked. Okay. So this is called as casual selfmasked attention mechanism. Okay. which hides the future words uh from the model. Now why encoders cannot be used for text generation? Okay, encoder understands the language. Then why cannot it generate the language? The question is the answer is very simple. The transformer encoder uses birectional attention. We have told that it reads the words from left to right and right to left and this attention mechanism is there each and every word you know it it connects to each and every word. So it

[11:27] is like left to right and right to left birectional attention mechanism comes into play in the encoder. Now if that happens when during your text generation that becomes a problem. How? See the cat sat on the mat as I was telling. So here these double-sided you know both ways arrows show that it is birectional attention mechanism is getting played in this uh sentence. Now what happens here every word attends to every other word. So every word there is no masking here every word. So the model sees all these words this side all these words from this side both the sides and the ma so if we use the encoder for text generation what happens is see our primary requirement in the text generation is casual mask self attention

[12:22] so that masking is impossible here because we are doing birectional you know attention mechanism is playing in the encoder so if encoder is used for texture generation during the prediction of the mat the encoder can already have attended to the mat because it was it has seen it when it is coming from right to left. So there is an information leakage that has happened. So that is the reason encoders cannot be used for text generation because that masking is not possible. Okay, masking of the casual uh masked attention is not possible in encoder because it is birectional attention. Okay. So that is the reason encoders cannot be used for language generation. Now

we will look into this auto regressive text

[13:09] generation by GPT. Okay. What exactly I mean? So I have a table here. Let's look into it one line at a time. Suppose our input to the model input to model is this one. So my initial prompt is the capital of France is. Now the model predicts based on this input the model predicts the word Paris. Now this goes and sits here in the second sentence. So the next prompt is the capital of France is Paris. Okay. After this one I tell the model to predict something. It predicts a full stop.

[13:50] Then this full stop comes and sits here. The capital of France is Paris. Full stop. Then I tell model to generate the next token. What it generates is it. So this now comes here and sits here. Okay. Like all the all the places also Paris came and sat here. So what happens is the capital of France is Paris. Full stop. It then I have then what will the model model predict? What is the next word prediction? So these are the next word predictions basically. Okay. Next word prediction.

[14:33] Okay. Then after it it comes here and the model predicts it is. Then is also comes here. The capital of France is Paris. It is it predicts known. Then that comes here. Capital of France is Paris. It is known what is the next word for. This for comes here. The capital of France is Paris. It is known for. The next word is I. Uh the And then this comes here. So the capital of France is Paris. It is known for the next word predicted by the model is Eiffel. And then Eiffel comes here. And the capital of France is Paris. Full stop. It is known for Eiffel. The next word predicted by the model is tower. So the final output is the capital of France is Paris. It is known for the Eiffel Tower. So you see that the

[15:28] initial prompt was this one. The capital of France is after that it has generated this much of text and the way it has generated is one token at a time and that goes and sits in the input adds to the input sequence already that is given to the model and then it predicts the next word. Okay. So this process is called as the auto regressive text generation. Okay. So that is the process. Then we will look into the GPT learning framework. How does the learning happens in GPT? So we have large unlabeled text data large corpus very big corpus we have and then there is a generative pre-training which is the language modeling happens here. So there is here also there is a pre-training and a fine-tuning. So this is pre-training is

[16:19] the stage one of the learning framework and it learns the grammar semantics useful reasoning pattern from the text. Okay. So it learns grammar, it learns semantic, it learns the reasoning pattern what should come after what. Okay, these kind of reasoning or logical things are learned during the pre-training steps and then we get the pre-trained GPT model. Okay, and then it goes for supervised fine-tuning. So this pre-training as we have already told before also usually pre-training a self-supervised because you are giving unlabeled data. You are not supervising anything. It is a self-supervised one.

[16:59] The model automatically segregates it and makes a you know internal you know labeling for itself to learn. So it is a self-supervised one and then the finetuning is a supervised finetuning where you use labeled data set. The data set will be smaller much smaller than your pre-training data set because your finetuning is basically for making the model ready for specific tasks. Okay. So this is the stage two of your learning framework and it needs very little task specific modifications. We will see what kind of modifications are you know required. So there that we will see and then we get a task specific model.

[17:43] Okay we get task specific model. Okay. So that is uh what is in the uh GPT learning framework. Fine. And now we will see what happens in the generative pre-training self-supervised uh process. So pre-training enables GPT to learn a universal presentation of language predicting the next word from massive amount of unlabeled text. So this is self-supervised learning. Okay. So this is self-supervised learning as I have already told. So the input is the earth revolves around the sun. The target is earth revolves around the dash. The target is sun. Artificial intelligence is transforming dash. The the target word is industries. So like this what happens is the model repeats this prediction task billions of times

[18:37] gradually learning the statistical structure of the language. The model repeats this prediction that means multiple times you give this type of task to the model to do. What will the model learn? the model will learn the statistical structure of language. It will learn that one. Okay. So that is what is happening in a pre-training purpose. Now this learns grammar, vocabulary, context understanding, long range dependencies, so on and so forth.

[19:10] Okay. And finetuning once again it is supervised. You have large unlabeled text. Then this goes for pre-training the model. You have the pre-trained large language model because the model is trained on a huge amount of corpus. We call it as large language models. Then we use this pre-trained LLM for label task for uh sorry for fine-tuning. Now for that fine-tuning we have labeled data for a specific task and we fine-tune the LLM. Okay. So for question answering we have a specific question answering say in the um sports domain question answer uh in the education domain. So you have specific uh literature you have specific documents from the sports domain or from the education domain and then you use

[20:02] that data set for training your pre-trained GPT. you take the pre-trained GPT then you train it and with this specific labeled data set and then you fine-tune the LLM. Okay. So in fine-tuning the thing that I want you all to have a notice is the pre-trained weights are updated using supervised learning. So there is no model training from the scratch. we take the weights that are already uh you know given by the pre-trained model and then we update those weights with this specific data set. Okay, when we are try training with this specific data set. Okay, and no need to train the model from the scratch that reduces your time complexity, the computational power

requirement, everything. Okay. Now, the

[20:54] fine-tuning task includes text classification, question answering, natural language inference, semantic similarity, so many things. Okay. So, we will have a look into this architecture. Okay. And we will see what is this GPT architecture the decoder only transformer. I have taken this image from the uh paper and we will see what exactly it is uh showing. Okay. We will try to understand that one. So we have this 12 layers of decoder and then we have this text and position embedding which is going inside your decoder one layer inside that one you have the mast multi- self attention okay and then you have the layer normalization fit forward then again layer normalization these are the normal layers that are

[21:55] present in the decoder. Now one thing you will uh you know um you should um notice here is there in the normal decoders we had another multi head self attention which was between the encoder and the decoder but that is missing here. If you go back to your our transformer architecture, the normal one which we studied there was an encoder decoder multi head you know uh self attention layer in the after this layer normalization and you know in between and before the feed forward so there was a uh another layer of multi head self attention which was the attention between the encoder and the decoder but in GPT we are not using the encoder so what happens that self attention head is missing. So we have here the masked

[22:47] multi self multi head self attention layer which is used for masking the future tokens. So the model will see only the tokens which have come already and it will the it will not see the tokens which are coming in the future. Okay. So that is the function of this masked multi- self attention. Then we have the layer normalization and then we have the skip connections and uh the feed forward that we have already seen in the this one what so this is the entire decoder architecture.

[23:22] Okay. So this is the decoder and then you see here there is one text prediction and task classifier. So what is this one? So this text prediction is basically the objective of your pre-training. So transformer architecture and training objective. So your training objective in the pre-training part is basically your text prediction. So each and the I have already told that the GPT uh is pre-trained using the process of next word prediction. So it is predicting the next word by that it is learning. So this is the you know the pre-training part and this is uh this is basically transformer architect training objective of pre-training okay and this is pre pre-training and then this part is basically the task

[24:19] classifier which actually is telling what is your downstream task. So the text prediction is your pre-training objective. Okay. And then this side gives you this entire site gives you input transformations for fine-tuning on different tasks. So this part we have already understood. This is basically your decoder and decoder is learning the text predictions using the pre-training. Okay. So this is done. Now there

is these are the downstream tasks which is we are calling it as class task classifier. That means it is classifying which type of downstream task you want to do. Okay. So there are couple of examples here of downstream tasks which are done. Classification entailment similarity multiplechoice questions and

[25:15] answers. So we will see what exactly happens in this part. Okay. So this I have taken from the original paper. So you can have a look into the paper also. So you see in classification suppose my downstream task is classification. Okay that means I want I have some input to the model and I want the sentiment classification to be done based on that one something else to be written. Okay. So that is my classification. So there is a start token which says the beginning of the sequence of tokens beginning of sequence. Okay. So and there is the text goes after that one whatever is my big thread text and then there is an extract. So end token whose final hidden state is used for prediction. So basically this is the

[26:03] start token and this is the end token whose final hidden state is used for the prediction. So it is used for the prediction and then this one goes to the transformer which is actually a pre-trained transformer. After that there is a linear layer and this linear layer will give you the output that you are requiring. Okay. So this will give you say say positive or negative sentiment. Okay. Whereas in entailment you have the start token then you have the premise and the deliminitor uh and then there's a hypothesis. So what is this entailment basically? So you have a premise a sentence and then you have a hypothesis. So if there is those the premise and the hypothesis they are you know aligning to each other that is we

[26:52] call it as entailment. If they're contradicting each other then we call it as you know contradiction and if it is um you know neither contradicting nor or you need more information about the premise or the hypothesis to uh tell whether it is contradicting or it is aligning then we call it as neutral. Okay. So what happens is in this case also you give the actual premise what exactly is your context or give the first sentence and then there is a delimiter. Delimiter is basically a separator and then you give the hypothesis. Okay, we will just see an example and then there is an extract token and this entire thing goes to the transformer and then it goes to the linear layer. So what happens is suppose

[27:40] the but birds can fly birds can fly is your premise and the hypothesis is sparrows can fly. So these two are aligned. So when you are giving the entailment neutral or contradiction so it is entailment will come into picture. Okay. So this is how the entire you know uh understanding or getting the context of the uh sentence or the paragraph is uh processed in the decoder in the GPT. Okay. Similarly here you have an example of similarity of two sentences. Say text one and text two. These are two text which you have given to the uh model with the start and the extract token and a delimiter in between which is a separator and then text two and then text one and you want to see what is the similarity between them and these two

[28:36] goes to the transformer architecture and then the transformers you know sees both the things and then it goes through a linear layer. So this is basically these linear layers are added layers for the finetuning we tell right class specific layers are added for finetuning. So that is what has happened here which is the linear layer and then the output will be similar or not similar. Okay. Similarly in multiple choice questions you see there is some context which is given there is some answers that are given answer one answer two answer n in between the start and the extract and these are fed to the transformer and these are linear layers are there for each of these answers options with the context and what we get is each produces

[29:21] a score. So if the context and the answer one are correct pair it produces a high score. If the context and the answer one is not correct pair then it produces a low score. The answer with the highest score is selected. Okay. So this is the way your architecture GPT architecture works. Okay. Then we will look into how these large language models or GPT1 and GPT2. I have in the initial part of my lecture itself. I have told there are so many you know uh uh upgradations or v not variants upgradations or versions of GPT that is coming and is still evolving and it has come to a very very you know technologically advanced stage. Now we will see uh a large language model what exactly we mean when we tell a large

[30:15] language model. So we keep on telling right these large language models are used for various kind of applications. So when we tell the word large language model what exactly we mean? So large language model is a transformer based network trained on massive text using selfsupervised next token prediction. GPT is one of its kind and it learns the language part pattern and we have a single pre-trained model that can be adapted for different NLP tasks. So we have seen the in our previous slide how that single pre-trained model has been you know um um has been uh you know with a separate layer that has been added linear layers task specific layers have been added at the last end to make it some some some model will be very

[31:14] specific to question answering some models will be very specific to sentence um sentiment analysis. Some will be very specific to text classification so on and so forth. Okay. So, GPT is a large language model. Okay. So, then we will see at least the difference between GPT1 and GPT2 the later you know later as the years evolved and we got so many you know GPT3 3.5 4 so many you know advancements are there. So let us look into GPT 1 and GPT2. So both a decoder only transformer here in GPT1 number of parameters were 17 117 million where it is 1.5 billion. So if you see even the successive GPT models there the number of parameters are you know it is increasing and the training data size also increases here it was initially it

[32:07] was 5GB now here it is in GPT2 it was 40GB. So high quality web pages the context length the number of tokens that can be processed that was 512 tokens in GPT1 1,200 1,024 in GPT2 and this context length also keeps on increasing as with the uh next uh you know uh subsequent models also the vocabulary on which the GPTs

were trained the initial one GPT1 was 40k K uh and this was 50k bite pair encodings and then training object was same 10 training objective was next token prediction. Okay that is same. So the same thing that is there is the decoder architecture and the next token prediction which is the training objective but the parameters the training data the context length the vocabulary all have increased from one

[33:03] to two and you know in the subsequent models to enhance the performances. Okay. And then multimodal things have been introduced. Uh so text to image to text, text to images and so many you know multimodal uh approaches have come into picture with the various advancements. Okay. Now GPT is extremely doing good but there are some limitations. One is hallucination. That means it makes up a story and gives it with um you know makes up a story and gives it as an output without that story may not be actually you know um that is not the correct information. So it is actually making out making up some information. So this is called as hallucination. So hallucination is basically whatever information is not

[33:55] correct your model is making up information in that direction and giving it as an output. Okay. And it gives it with a lot of confidence also. Then there can be biases in the output. If your training data is having some bias, suppose you are training your data with training your model with a data which is telling 95% of the time you are telling um you know the president of a country is you you are telling it is he or a man or a gentleman. Okay. So then the model becomes biased and it learns if it is the president or any leader you know of that category stature they have to be a male. So these are the biases which is not correct. So these are the biases that may come in the model's prediction.

[34:47] Together with that one there is a large computational cost that is involved. Context window limitations are there to a certain extent of tokens. the uh the model can process at a time. Prompt sensitivity. If you give if you change the prompt to the model a little bit, the predictions may change drastically. Okay. So this is called as prompt sensitivity. Knowledge cutff for the base models particularly. So the knowledge cutff is basically if you train these are majorly for the base models. So if you train if you have trained your model till 2022 or 23 and if you are asking something of 25 or 26 the year 25 or 26 to 2025 26 it may not be able to give information of course uh that that that things have been removed

[35:39] now but yeah in the base models this limitation was there and of course there are some safety concerns if it can produce some you know uh information which is not safe. Okay. So, safety concerns are there if it is prompted wrongly. So, these are the limitations of GPT and as I was telling this prompt sensitivity or prompting is very important to get proper predictions from your model from the large language models. Okay. So prompt engineering builds on GPT by learning how how we have to communicate effectively with LLMs for getting an output that we want we expect to get. Okay. So prompting technique is very important in cases of LLMs. Okay. So

with this one I come to the uh end of today's session.

[36:34] Before concluding I will tell let us you know uh just have a quick look into what we have gone through in today's session. So we have seen what was the limitation in um the earlier language models like statistical you know models engram models and other things. Uh what was the problem? So once again the problem was uh you know labeled text data was not available to a lot of you know time and uh there was a there was requirement of specific NLP uh for specific NLP task you required model specific models to be trained which was expensive. So from there GPT also came into picture in the same year as birth. The main focus of GPT was in the context generation, sentence generation not uh and unlike BERT which

[37:24] was more focused towards understanding the language and then we have seen that the GPT is a decoder only architecture and how it processes the entire you know the the learning framework is divided into two steps the pre-training and the fine-tuning. We have seen how the pre-training happens here. The m the main objective of pre-training here is next token prediction. So through that process only it learns the pre-training happens and then once we get the pre-trained models it is uh it is uh there is one extra layer that is added to the you know architecture for making that model specific for a particular application like question answering. It can be sentiment analysis. It can be natural language inferences. Okay. So on

[38:16] and so forth. So you have a base pre-trained model and then you are changing some things in the architecture little bit not some things but only in the last layer one additional one one layer and then you train it with some supervised or labelled data set on a specific task as I was telling if it is question answering on uh say history you you uh train the model in the fine-tuning process with all the documents that are relevant to history of a uh you know of a country and then you train it and predict the answers from there. So that is the main work of GPT and uh when it is this kind of um you know language models when they are um when they are trained on massive corpus uh with a huge amount of

[39:09] parameters and massive corpuses. So they they become the large language models which we use uh you know now nowadays for everything we use these large language models and then we have seen that okay these large language models what are the limitations what can be the limitations of large language models one is the hallucination one is the biases in the output then u then you can have context length limitations so these are few limitations prompt sensitivity that is one of the very important things. So if you prompt it in one way you will get one type of prediction or one type of text generation. If you prompt it in some other way you get a different type of prediction. Okay. Uh prediction means

[39:54] text generation in this case. Now this prompting technique is very very important and in my next uh lecture uh it is completely on prompt engineering. Uh

so we will look into the prompt engineering techniques. What are the techniques by which if we prompt an a large language model we can expect to get uh generated text which is up to our requirement okay up to our expectation as per our expectations. So with this one I conclude uh today's session and the next session is on prompt engineering and uh till we meet for the next session take care please go through the um original paper that I have given the reference uh because you will get a lot of information about GPT um in details. So please go through that one

[40:43] um and uh please keep learning, happy learning and thank you all.